

ET4351 Labs Report - Lab 1

Student Name	NetID	Course Code
Tyukov, Daniel	datyukov	ET4351

Lab 1: Project Baseline with PicoSoC

Question 1.1

(a) How does PicoSoC load the firmware?

The testbench (et4351_tb.sv) instantiates the et4351 top-level module (the DUT) and a spiflash module. During initialization, spiflash.v loads the firmware hex file into its internal memory array using `$readmemh(firmware_file, memory)`, where the firmware path is passed via the `+firmware=` plusarg. At reset, the PicoRV32 CPU inside PicoSoC begins fetching instructions starting from `PROGADDR_RESET = 0x0010_0000` (1 MB into the flash address space). The spimemio module in picosoc.v handles SPI flash reads: when the CPU issues a memory access in the flash range (address $\geq 4 * \text{MEM_WORDS}$ and $< 0x0200_0000$), spimemio drives the SPI protocol signals (csb, clk, io0-io3) to the external spiflash model, which responds with the requested data bytes. The flash model supports multiple SPI modes (standard, dual, quad, DDR) for increasing throughput.

(b) How does PicoSoC display "Hello, World!" on the terminal?

The C firmware (hello.c) calls `init_uart()` which sets the UART clock divider register at address `0x02000004` to 104, configuring the baud rate. Then `print_str("Hello, world!\n")` iterates over each character and calls `print_char(c)`, which writes the character to the UART data register at address `0x02000008` (`reg_uart_data`). In picosoc.v, writes to `0x02000008` are routed to simpleuart's `reg_dat_we` input. The simpleuart module serializes each byte into a UART frame (start bit, 8 data bits, stop bit) and shifts it out on `ser_tx` at the configured baud rate. The testbench (et4351_tb.sv) monitors `ser_tx` with an always block that detects the start bit (negedge), samples 8 data bits, and uses `$write` to display each character on the QuestaSim console. When `print_char(-1)` sends a value > 127 , the testbench terminates the simulation via `$finish` after printing the cycle count (26,554 cycles).

Question 1.2

The dummy accelerator (accelerator.v) is a counter-based module mapped to address range `0x0300_0000`. It has 4 memory-mapped registers: GPIO (`0x03000000`), CSR (`0x03000004`), Input (`0x03000008`), and Output (`0x0300000C`). The accelerator is **launched** by the CPU writing to the CSR register: bit 0 controls reset (active-high), bit 1 enables the accelerator, and bit 2 is a read-only done flag. The memory interface responds when `iomem_valid` is asserted and `iomem_addr[31:24] == 0x03`. Reads and writes to the 4 registers are handled byte-by-byte using `iomem_wstrb`.

The **handshake** mechanism works as follows: the CPU writes the target count to the Input register, then sets the enable bit in CSR. The accelerator increments an internal counter each clock cycle until it reaches the target value. Upon completion, it copies the counter value to the result register and sets the `valid_out` flag, which is reflected in CSR bit 2. The CPU polls CSR bit 2 and reads the Output register when done. This is a simple polling-based handshake: the CPU starts the accelerator via CSR and waits for the done flag.

Question 1.3

In `dummy_accel.c`, the PicoRV32 core interacts with the accelerator through memory-mapped I/O registers defined as volatile pointers. The interaction follows these steps:

1. **Initialize:** All four registers (GPIO, CSR, Input, Output) are written to `0x00000000`. This is a write to addresses `0x03000000-0x0300000C` via the bus. In `accelerator.v`, these writes update `iomem_accel[0..3]`.
2. **Reset:** The CPU sets CSR bit 0 (`reg_csr |= MASK_CSR_RESET`) then clears it. In hardware, `reset_accel = iomem_accel[1][0]` resets the counter, result, and `valid_out` to 0.
3. **Set Input:** `reg_input = 0x00012345` writes the target count (74,565 decimal) to `iomem_accel[2]`, which is assigned to `count_dest`.
4. **Start:** `reg_csr |= MASK_CSR_START` sets bit 1, enabling the accelerator (`enable_accel`). The counter increments each cycle.
5. **Poll:** `while(!(reg_csr & MASK_CSR_DONE))` repeatedly reads CSR. Each read triggers a bus transaction; `accelerator.v` returns `iomem_accel[1]` with bit 2 reflecting `valid_out`.
6. **Read Output:** `reg_output` reads `iomem_accel[3]` (= result = `0x00012345`). The value is printed via UART, producing "Accelerator Output: 0x00012345".

Question 1.4

In Task 3, the `spiflash.v` model for the flash memory is modified compared to Tasks 1/2. The key difference is the memory declaration and an additional `$readmemh` call:

```
reg [7:0] memory [1*1024*1024 : 5*1024*1024-1]; declares a 4 MB flash starting at byte
address 0x00100000 and ending at 0x004FFFFF.
```

The firmware (.text section) is loaded first via `$readmemh(firmware_file, memory)`, occupying the lower portion of flash starting at `0x00100000`. Then the twiddle factors and signal samples are loaded via `$readmemh("../firmware/fft_data.hex", memory)`. According to the lab documentation (Figure 9), the signal samples are stored at the end of flash starting from address `0x004F0000`. Each sample occupies 4 bytes (32-bit integers) stored in little-endian byte order. The twiddle factors (precomputed complex exponentials) and the input signal samples are both placed in `fft_data.hex` by the `prepare_fft.py` script, which runs during compilation. The C code reads these values from flash using memory-mapped read accesses through the SPI interface, the same mechanism used for instruction fetches.

Question 1.5

The FFT latency results for different GCC optimization levels (32-point FFT, all verified correct):

Optimization Level	FFT Runtime (cycles)	Total Latency (cycles)	Speedup vs -O0
-O0	1,412,441	3,517,190	1.00x
-O1	264,711	1,331,445	5.33x
-O2	259,390	1,294,154	5.45x

The jump from -O0 to -O1 yields a dramatic ~5.3x speedup in FFT runtime. This is because -O0 performs no optimization (every variable is stored to and loaded from the stack), while -O1 enables register allocation, constant folding, dead code elimination, and basic loop optimizations. Moving from -O1 to -O2 provides a marginal further improvement (~2%), as -O2 adds instruction scheduling, loop unrolling, and more aggressive inlining. The relatively small -O1 to -O2 gap suggests the FFT computation is already well-optimized at -O1 and is limited by memory access latency (SPI flash reads) rather than instruction count alone.

This report is individual and formative. I certify that I have done this report individually.